

Assignment One

EE446

Report

Part 1- Python Notebook

Possibility of Further Model Size Reduction

Can you further reduce the model size beyond the smallest model obtained in parts (b), (c), or (d), without sacrificing significant classification performance?

1) Analyze and compare the results from previous parts: Which model had the smallest size? Which performed best?

Comparing the model sizes from different parts. In part b after performing 3 types of quantisations model size for int 8 was smallest of 5.74 KB . The quantised float 16 and dynamic model had highest accuracy of 90.74%

2) Propose a strategy that combines or enhances techniques learned so far.

Pruning pushes many weights to zero, but in FP32 format those zeros still occupy 4 bytes each, so the raw file size barely changes. So a strategy is to first strip the pruning wrappers, remove the masks, and then apply Float16 quantization, which halves the storage precision from 32-bit to 16-bit floats. I am doing light pruning of 30% instead of aggressive pruning of 70 percent to preserve accuracy on this small model.

3) Implement your proposed solution.

I have put the code in the notebook.

4) Evaluate the resulting model using both:

- ★ TFLite model size (in KB)

The model size is 9.09 KB

- ★ Classification performance (accuracy and report)

Pruning 30% + Float16 Quantization
Model size: 9.09 KB
Accuracy: 0.6481

Classification Report:

	precision	recall	f1-score	support
0	0.68	1.00	0.81	19
1	0.62	0.76	0.68	21
2	0.00	0.00	0.00	14
accuracy			0.65	54
macro avg	0.43	0.59	0.50	54
weighted avg	0.48	0.65	0.55	54

5) Justify your results:

If further size reduction is not possible without major loss of accuracy, explain why.

The baseline model is already very small. With so few parameters, each weight carries significant information for classification. Aggressive pruning at 70% sparsity, dropped accuracy to 55% because removing that many weights from an already small network eliminates critical learned information. Similarly, INT8 quantization alone dropped accuracy to 64% due to proportionally larger rounding errors. Combining both techniques would compound these losses, pruning removes important weights, and INT8 then roughly approximates the remaining ones, leading to even greater accuracy loss. Larger models perform well after these techniques because they have more dead weights but this model does not.

If you succeed in reducing the size further, highlight what change made the biggest difference.

Float16 quantization alone made the biggest difference, reducing the model from 14.07 KB (FP32) to 8.95 KB, a 36% reduction, while maintaining about 90% accuracy. It halves the storage precision from 32-bit to 16-bit floats, that is enough precision for a small model's weights without significant rounding errors. Adding 30% pruning on top did not further reduce size because Float16 stores zeros at the same cost as any other value, so zeroed-out weights from pruning had no compression benefit. For this small model, the quantization step alone did size reduction.

Part 2 - Edge Impulse

1) [Dis] (5 pts) What types of "Learning Blocks" are available under the free-tier Edge Impulse account (e.g., supervised learning—classification and regression, unsupervised

learning—clustering, self-supervised learning, etc.)? For each type, provide at least one example model that can be implemented using the free tier.

For supervised learning, the Classification (Keras) block categorizes inputs into discrete classes such as movement types model (walking, running, idle) from accelerometer data, while the Regression block predicts continuous values like temperature from sensor readings. Object Detection blocks like FOMO detect and locate objects in images on TinyML devices. For unsupervised learning, Anomaly Detection (GMM) and Anomaly Detection (K-Means) learn real data patterns and identify clusters without labeled anomaly samples, useful for detecting abnormal vibrations in machinery like the fan vibration model. Visual Anomaly Detection (FOMO-AD) extends this to image-based inspection tasks like spotting surface defects. For transfer learning, the Transfer Learning (Images) block fine-tunes pre-trained models like MobileNet on small custom datasets for image classification, and Transfer Learning (Keyword Spotting) for voice command recognition with audio data.

2) [Dis] (5 pts) List all the types of layers that can be used under the “Classifier” section in the free-tier Edge Impulse. Briefly explain the purpose of each layer identified.

Layers under the classifier are as follows:

- ★ Dense layer – A fully connected (FC) layer in a DNN where all neurons are connected to each other. It is mainly used for learning patterns and making the final classification.
- ★ 1D Conv layer – Learns features across a single dimension from a vector or sequential data such as sensor or audio data.
- ★ 2D Conv/Pool layer – Learns features across two dimensions from array-like data such as images. The convolution part extracts features, while pooling reduces the size of the feature map and helps reduce overfitting.
- ★ Reshape layer – Used to transform data into a different shape or dimension so it can be used correctly by other layers in the network.
- ★ Flatten layer – Usually done after convolution blocks to convert multi-dimensional data into a single vector before passing it into fully connected layers.
- ★ Dropout layer – Randomly turns off some neurons during training to help prevent overfitting and improve generalization.

3) [Dis] (5 pts) What types of model compression options are available under the “Deployment” section in the free-tier Edge Impulse? List all available methods.

The model compression options available under the Deployment section in the free-tier of Edge Impulse are:

- ★ Float32 quantization – Uses 32-bit floating point value representation. It gives higher accuracy but requires more memory and computation.
- ★ Int8 quantization – Compresses the model into 8-bit integer value representation of weights and activations, reducing model size, making inference faster and is more efficient for TinyML devices.

4)[Dis] (5 pts) What input modalities are supported for synthetic data generation in the free-tier version of Edge Impulse? For each modality, list the synthetic data generation

methods/algorithms provided. Briefly explain why synthetic data is important in machine learning applications.

The input modalities supported for synthetic data generation in the free-tier of Edge Impulse are:

- ★ Image modality
 - Black Forest Labs Pro Synthetic Image Generator – Uses Replicate.com’s API for the Flux Pro model from Black Forest Labs to generate high-quality synthetic images with good prompt following and image detail.
 - DALL·E 3 Synthetic Image Generator – Uses OpenAI’s DALL·E 3 model to generate synthetic images for projects.
- ★ Audio / Voice modality
 - Whisper Synthetic Voice Generator – Uses OpenAI’s text-to-speech model to generate synthetic voice samples. This is useful for building keyword spotting models for custom words or phrases.

Synthetic data is important in machine learning because sometimes real-world data is limited or some classes have very little data compared to others. Synthetic data helps increase the dataset size and improve model performance. For example, in image datasets we can apply transformations such as horizontal flipping, rotation, or scaling, while in audio datasets we can add noise or change frequencies to create more training samples and improve generalization.